



**WYDZIAŁ MATEMATYKI
i INFORMATYKI**

Uniwersytet Łódzki

Szymon Pawłowski

Nr albumu: 408720

**Predykcja popularności piosenek na
podstawie ich cech audio - projekt uczenia
maszynowego**

**Praca inżynierska
na kierunku Analiza danych**

Praca wykonana pod kierunkiem

dr Marty Lipnickiej

Katedra Sztucznej Inteligencji i Analizy Nieliniowej

Łódź, 2026

Słowa kluczowe: analiza danych, uczenie maszynowe, lasy losowe, predykcja

Title in English: Song popularity prediction based on audio features - a machine learning project

Keywords: data analysis, machine learning, random forest, prediction

Spis treści

1	Wstęp	7
1.1	Tematyka oraz cel pracy	7
1.2	Zakres wkładu autorskiego oraz struktura projektu	8
1.2.1	Transformacja problemu badawczego	8
1.2.2	Opracowanie autorskiego potoku przetwarzania danych (Data Pipeline)	8
1.2.3	Optymalizacja hiperparametrów modeli	8
1.2.4	Wielokryterialna ewaluacja i analiza porównawcza	9
1.2.5	Serializacja modeli i przygotowanie do wdrożenia	9
1.3	Metodyka gromadzenia danych i literatura źródłowa	9
2	Podstawowe pojęcia	10
2.1	Teoria uczenia maszynowego	10
2.2	Metryki oceny jakości modeli	11
2.3	Algorytmy klasyfikacji	12
2.3.1	Analiza Składowych Głównych (PCA)	13
2.3.2	Algorytmy oparte na odległościach	13
2.3.3	Modele liniowe	14
2.3.4	Metody zespołowe (Ensemble Methods)	14
2.3.5	Sztuczne Sieci Neuronowe	15
3	Część główna - projekt i implementacja	16
3.1	Środowisko programistyczne i narzędzia	16
3.2	Charakterystyka i przygotowanie danych	17
3.2.1	Struktura zbioru danych	17
3.3	Analiza korelacji i charakterystyka problemu	18
3.3.1	Implikacje dla doboru modelu	20
3.3.2	Transformacja problemu i inżynieria cech	20
3.3.3	Normalizacja danych	21
3.4	Implementacja modeli uczenia maszynowego	21

3.4.1	K-Najbliższych Sąsiadów (KNN)	22
3.4.2	Random Forest (Las Losowy)	22
3.4.3	XGBoost i Sieci Neuronowe (MLP)	22
3.5	Przygotowanie do wdrożenia	22
4	Rozdział badawczy - analiza wyników	24
4.1	Analiza porównawcza modeli i uzasadnienie wyboru	24
4.1.1	Zestawienie wyników eksperymentu	25
4.1.2	Analiza wydajności i wybór modelu końcowego	26
4.2	Zmiana podejścia: Regresja vs Klasyfikacja	28
4.3	Wyniki klasyfikacji za pomocą modelu Random Forest	29
4.3.1	Szczegółowy raport klasyfikacji	30
4.4	Analiza ważności cech (Feature Importance)	32
5	Studium przypadku (Case Study)	33
5.1	Przypadek 1: współczesny przebój radiowy	33
5.2	Przypadek 2: klasyk rockowy (Analiza błędu)	35
5.3	Przypadek 3: utwór niszowy	36
6	Podsumowanie i wnioski	38
6.1	Stopień realizacji celów pracy	38
6.2	Wnioski merytoryczne	38
6.3	Ograniczenia rozwiązania i kierunki dalszych prac	39

Spis rysunków

3.1	Korelacja cech audio ze zmienną song_popularity	19
4.1	Macierz pomyłek dla algorytmu KNN (k=25) po redukcji PCA	27
4.2	Krzywa ROC dla modelu Random Forest	29
4.3	Wizualizacja macierzy pomyłek dla modelu Random Forest	31
4.4	Wpływ poszczególnych cech audio na predykcję popularności	32
5.1	Wynik predykcji aplikacji dla utworu „I Like It”	34
5.2	Wynik predykcji aplikacji dla utworu „Wind Of Change”	35
5.3	Wynik predykcji aplikacji dla utworu niszowego	37

Spis tabel

2.1	Schemat macierzy pomyłek dla klasyfikacji binarnej.	11
2.2	Systematyka zastosowanych algorytmów klasyfikacji.	12
3.1	Liczebność oraz udział procentowy klas w zmiennej decyzyjnej.	21
4.1	Porównanie skuteczności badanych modeli na zbiorze treningowym i testowym.	26
4.2	Raport klasyfikacji (Classification Report)	30
4.3	Wagi najważniejszych cech (Top 5)	32

Rozdział 1

Wstęp

1.1. Tematyka oraz cel pracy

Współczesny przemysł muzyczny podlega intensywnej transformacji cyfrowej, w której platformy strumieniowe (ang. streaming services) zdominowały tradycyjne kanały sprzedaży (takie jak fizyczne nośniki danych) oraz tradycyjne media nadawcze. Serwisy takie jak Spotify czy Apple Music generują codziennie milionowe wolumeny danych, rejestrując zarówno szczegółowe metadane utworów oraz ich cyfrowe cechy akustyczne, jak i precyzyjne informacje o zachowaniach użytkowników (np. historię odtworzeń, pomijanie utworów, tworzenie playlist). Tak bogate zbiory danych stanowią fundament dla zaawansowanej analityki danych (ang. data analytics) oraz systemów uczenia maszynowego. W tym kontekście możliwość predykcji przyszłej popularności utworu muzycznego - realizowana na podstawie ekstrakcji jego parametrów audio oraz metadanych - staje się kluczowym narzędziem wspierającym podejmowanie decyzji biznesowych przez wytwórnie płytowe, producentów oraz artystów.

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie, implementacja oraz ewaluacja modeli uczenia maszynowego (ang. Machine Learning) służących do predykcji popularności utworów muzycznych. W ramach badań przeprowadzona zostanie analiza porównawcza wybranych algorytmów klasyfikacji i regresji. Celem tego procesu jest wyłonienie modelu, który najskuteczniej odróżnia utwory o wysokim potencjale rynkowym (tzw. „hity”) od pozostałych kompozycji.

1.2. Zakres wkładu autorskiego oraz struktura projektu

Kluczowym elementem niniejszej pracy jest realizacja pełnego potoku przetwarzania i analizy danych (ang. Data Science Pipeline). Wkład własny autora obejmuje kompletny proces badawczo-programistyczny - od etapu preprocesingu danych, aż po przygotowanie produkcyjne modeli predykcyjnych.

Główne elementy wkładu autorskiego w pracy to:

1.2.1. Transformacja problemu badawczego

Zdefiniowano problem badawczy poprzez transformację zadania regresji w problem klasyfikacji binarnej. W tym celu zaimplementowano logikę decyzyjną opartą na z góry określonym progu popularności (wartość 0,70). Rozwiązanie to pozwoliło na jednoznaczne wyodrębnienie klasy docelowej (utworów o statusie „hitu”) i dostosowanie wyników analizy do realiów biznesowych przemysłu muzycznego.

1.2.2. Opracowanie autorskiego potoku przetwarzania danych (Data Pipeline)

Wytworzono dedykowane środowisko w języku Python z wykorzystaniem bibliotek pandas, matplotlib, seaborn oraz scikit-learn. W ramach potoku danych zaimplementowano procedury usuwania rekordów niekompletnych (ang. missing values), selekcji cech numerycznych oraz ich normalizacji za pomocą skalowania Min-Max. Etap ten zapewnił poprawność strukturalną danych wejściowych, wymaganą przez algorytmy uczenia maszynowego.

1.2.3. Optymalizacja hiperparametrów modeli

Przeprowadzono proces doboru hiperparametrów (ang. Hyperparameter Tuning) dla czterech zróżnicowanych struktur matematycznych: Lasów Losowych (Random Forest), K-Najbliższych Sąsiadów (KNN), algorytmu XGBoost oraz Sztucznych Sieci Neuronowych (MLP). Do przeszukiwania przestrzeni hiperparametrów wykorzystano metody GridSearchCV oraz RandomizedSearchCV, co pozwoliło na podniesienie efektywności predykcyjnej modeli w stosunku do ich konfiguracji domyślnych.

1.2.4. Wielokryterialna ewaluacja i analiza porównawcza

Dokonano oceny efektywności wytrenowanych klasyfikatorów w oparciu o zestaw metryk: dokładność (Accuracy), precyzję (Precision), czułość (Recall) oraz pole pod krzywą ROC (AUC). Wyniki zestawiono w formie tabelarycznej i graficznej. Przeprowadzona analiza porównawcza wykazała wyższą skuteczność metod zespołowych (Random Forest) w stosunku do pojedynczych modeli klasyfikacyjnych.

1.2.5. Serializacja modeli i przygotowanie do wdrożenia

Ostatni etap prac obejmował serializację finalnych modeli do formatu plików .joblib. Proces ten zabezpieczył struktury modeli wraz z ich wagami, stanowiąc bezpośredni fundament pod integrację z warstwą aplikacyjną (oprogramowaniem interfejsu użytkownika).

1.3. Metodyka gromadzenia danych i literatura źródłowa

Praca została przygotowana w oparciu o literaturę przedmiotu, dokumentację techniczną oraz zbiory danych dostępne publicznie.

Literatura naukowa: część teoretyczna pracy bazuje na polsko- i anglojęzycznych publikacjach dotyczących eksploracji danych (ang. *Data Mining*), statystyki oraz teorii uczenia maszynowego. Szczególną uwagę poświęcono pozycjom opisującym zasady działania algorytmów drzewiastych oraz sztucznych sieci neuronowych.

Dokumentacja techniczna: w procesie implementacji kluczowym źródłem wiedzy była oficjalna dokumentacja bibliotek środowiska Python, w szczególności Scikit-learn User Guide oraz dokumentacja biblioteki XGBoost. Źródła te posłużyły do poprawnego zaimplementowania parametrów modeli oraz zrozumienia mechanizmów ich optymalizacji.

Środowisko Programistyczne: całość prac programistycznych zrealizowano w środowisku Jupyter Notebook, co zapewniło transparentność i powtarzalność przeprowadzonych eksperymentów.

Dane źródłowe: podstawą analizy był zbiór danych zawierający parametry akustyczne utworów muzycznych [7].

Rozdział 2

Podstawowe pojęcia

2.1. Teoria uczenia maszynowego

W niniejszym podrozdziale przedstawiono teoretyczne fundamenty uczenia maszynowego oraz usystematyzowano aparat pojęciowy niezbędny do opisu procesów modelowania predykcyjnego wykorzystanych w pracy. Zdefiniowane poniżej terminy oraz metody oceny jakości modeli stanowią bazę merytoryczną, na której oparto realizację części projektowej (szczegółowe rozwinięcie zagadnień znajduje się w literaturze źródłowej patrz [1] lub [2]).

Definicja 2.1.1. *Niech X będzie przestrzenią cech (danych wejściowych), a Y przestrzenią etykiet (danych wyjściowych). **Uczeniem nadzorowanym** (ang. supervised learning) nazywamy proces poszukiwania funkcji $f : X \rightarrow Y$, która na podstawie zbioru uczącego $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ aproksymuje rzeczywistą zależność między danymi a ich etykietami tak, aby minimalizować błąd predykcji na nowych danych.*

W zależności od charakteru przestrzeni Y , wyróżnia się dwa podstawowe typy problemów uczenia nadzorowanego:

Definicja 2.1.2. *Jeżeli zbiór wartości wyjściowych Y jest zbiorem skończonym i dyskretnym (np. $Y = \{0, 1\}$), to problem nazywamy **klasyfikacją**. W przypadku, gdy $|Y| = 2$, mówimy o klasyfikacji binarnej.*

Definicja 2.1.3. *Jeżeli zbiór wartości wyjściowych Y jest zbiorem ciągłym (np. $Y = \mathbb{R}$), to problem nazywamy **regresją**.*

W kontekście niniejszej pracy, problem predykcji popularności utworu może być rozpatrywany zarówno jako problem regresji (przewidywanie wartości z przedziału $[0, 100]$), jak i klasyfikacji (określenie czy utwór jest "hitem").

2.2. Metryki oceny jakości modeli

Aby porównać skuteczność zaimplementowanych algorytmów, konieczne jest zdefiniowanie miar oceny jakości klasyfikatora. Podstawą do ich wyznaczenia jest tzw. macierz pomyłek (ang. confusion matrix).

Definicja 2.2.1. *Macierz pomyłek to tabela kontyngencji o wymiarach $N \times N$ (gdzie N to całkowita liczba klas), która służy do szczegółowej oceny wydajności modelu klasyfikacyjnego. Wiersze tej macierzy odpowiadają rzeczywistym klasom występującym w zbiorze danych, natomiast kolumny reprezentują klasy wytypowane (przewidziane) przez model.*

W celu ustrukturyzowania pojęć związanych z oceną klasyfikatora binarnego, strukturę macierzy pomyłek przedstawiono w tabeli 2.1.

Tabela 2.1: Schemat macierzy pomyłek dla klasyfikacji binarnej.

		Klasa przewidywana	
		Pozytywna (1)	Negatywna (0)
Klasa rzeczywista	Pozytywna (1)	True Positive (TP)	False Negative (FN)
	Negatywna (0)	False Positive (FP)	True Negative (TN)

Gdzie poszczególne komponenty oznaczają:

- **True Positive (TP)** – przypadki rzeczywiste dodatnie, które model poprawnie sklasyfikował jako dodatnie.
- **True Negative (TN)** – przypadki rzeczywiste ujemne, które model poprawnie sklasyfikował jako ujemne.
- **False Positive (FP)** – błąd I rodzaju; przypadki rzeczywiste ujemne, które model błędnie uznał za dodatnie.
- **False Negative (FN)** – błąd II rodzaju; przypadki rzeczywiste dodatnie, które model błędnie uznał za ujemne.

Definicja 2.2.2. *Dokładnością (ang. Accuracy) nazywa się stosunek liczby poprawnych predykcji do liczby wszystkich przypadków:*

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

Sama dokładność bywa myląca w przypadku zbiorów niezbalansowanych. Dlatego stosuje się również inne miary.

Definicja 2.2.3. Precyzję (ang. *Precision*) nazywa się stosunek poprawnych identyfikacji klasy pozytywnej do wszystkich wskazań tej klasy przez model:

$$Precision = \frac{TP}{TP + FP}$$

Definicja 2.2.4. Czulość (ang. *Recall*) określa się stosunek poprawnie wykrytych przypadków pozytywnych do wszystkich rzeczywiście istniejących przypadków pozytywnych:

$$Recall = \frac{TP}{TP + FN}$$

Definicja 2.2.5. (Miara F1) Miara *F1* (ang. *F1-Score*) stanowi średnią harmoniczną precyzji (*Precision*) oraz czulości (*Recall*). Jest to miara szczególnie użyteczna w przypadku oceny modeli operujących na zbiorach niezbalansowanych, ponieważ pozwala na jednoczesne uwzględnienie obu tych aspektów efektywności klasyfikatora:

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN} \quad (2.1)$$

2.3. Algorytmy klasyfikacji

W pracy badawczej wykorzystano zróżnicowane podejścia do problemu uczenia maszynowego, które można pogrupować w rodziny algorytmów ze względu na ich zasadę działania. Systematykę zastosowanych metod klasyfikacji przedstawiono w tabeli 2.2.

Tabela 2.2: Systematyka zastosowanych algorytmów klasyfikacji.

Rodzina algorytmów	Reprezentant w pracy
Oparte na odległościach	K-Najbliższych Sąsiadów (KNN)
Modele liniowe	Regresja Logistyczna
Metody zespołowe (Ensemble)	Random Forest, XGBoost
Modele koneksjonistyczne	Sztuczne Sieci Neuronowe (MLP)

Poniżej przedstawiono teoretyczne podstawy każdej z powyższych grup, co pozwoli na zrozumienie mechanizmów ich funkcjonowania oraz przesłanek wyboru do dalszych analiz. W pracy badawczej wykorzystano zróżnicowane podejścia do problemu uczenia maszynowego, obejmujące modele liniowe, metody oparte na sąsiedztwie, zaawansowane metody zespołowe (ensemble methods) oraz sztuczne sieci neuronowe. Poniżej zdefiniowano teoretyczne podstawy każdego z użytych algorytmów.

2.3.1. Analiza Składowych Głównych (PCA)

W przypadku pracy z wielowymiarowymi zbiorami danych cech audio, algorytmy oparte na odległościach (takie jak KNN) są wysoce podatne na tzw. przekleństwo wymiarowości (ang. *curse of dimensionality*). Aby zminimalizować to zjawisko, zastosowano technikę redukcji wymiarów przestrzeni cech.

Definicja 2.3.1. *Analiza Składowych Głównych* (ang. *Principal Component Analysis, PCA*) to liniowa technika statystyczna służąca do redukcji wymiarowości zbioru danych przy jednoczesnym zachowaniu jak największej ilości informacji (wariancji) zawartej w zmiennych pierwotnych. Matematycznie, algorytm dokonuje ortogonalnego przekształcenia współrzędnych, mapując dane do nowej przestrzeni, w której nowe zmienne - składowe główne (ang. *principal components*) - są wzajemnie nieskorelowane i uporządkowane według wielkości wyjaśnianej przez nie wariancji.

W eksperymencie algorytm PCA został skonfigurowany tak, aby liczba rzutowanych składowych głównych gwarantowała zachowanie dokładnie 95% skumulowanej wariancji wejściowego zbioru danych. Pozwoliło to na znaczące zmniejszenie złożoności obliczeniowej modelu KNN bez utraty kluczowych właściwości informacyjnych sygnału audio.

2.3.2. Algorytmy oparte na odległościach

Do tej grupy należy algorytm K-Najbliższych Sąsiadów (KNN). Jego działanie opiera się na założeniu, że obiekty o podobnych cechach znajdują się blisko siebie w przestrzeni wektorowej. Kluczowe jest tutaj pojęcie metryki.

Definicja 2.3.2. Niech $x, y \in \mathbb{R}^n$. **Odległością Euklidesową** nazywa się funkcję $d(x, y)$ określoną wzorem:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2},$$

gdzie x_i oraz y_i to wartości i -tej cechy dla analizowanych obserwacji, natomiast n to całkowita liczba cech w wielowymiarowej przestrzeni danych.

Definicja 2.3.3. **K-Najbliższych Sąsiadów (KNN)** to algorytm nieparametryczny, który klasyfikuje nowy obiekt na podstawie klasy dominującej (głosowania) wśród k obiektów ze zbioru uczącego, dla których odległość d od badanego punktu jest najmniejsza.

2.3.3. Modele liniowe

Jako punkt odniesienia (baseline) dla bardziej złożonych metod zastosowano Regresję Logistyczną.

Definicja 2.3.4. *Regresja logistyczna to model probabilistyczny stosowany w problemach klasyfikacji binarnej. Model ten prognozuje prawdopodobieństwo przynależności warunkowej do klasy pozytywnej ($Y = 1$) dla danego wektora cech $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ za pomocą funkcji sigmoidalnej (logistycznej) $\sigma(z)$:*

$$P(Y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.2)$$

gdzie z stanowi liniową kombinację cech wejściowych wraz z wyrazem wolnym (odcięciem):

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n = \boldsymbol{\beta}^T \mathbf{x} \quad (2.3)$$

przy czym $\boldsymbol{\beta} = [\beta_0, \beta_1, \dots, \beta_n]^T$ oznacza wektor parametrów (wag) modelu, które są szacowane w procesie uczenia.

Funkcja sigmoidalna $\sigma(z) : \mathbb{R} \rightarrow (0, 1)$ posiada charakterystyczny kształt litery „S” i mapuje dowolną wartość rzeczywistą na przedział otwarty $(0, 1)$, co pozwala na bezpośrednią interpretację jej wyniku jako prawdopodobieństwa.

2.3.4. Metody zespołowe (Ensemble Methods)

Metody te łączą wiele słabszych estymatorów (zazwyczaj drzew decyzyjnych) w celu uzyskania modelu o mniejszej wariancji i lepszej zdolności generalizacji.

Definicja 2.3.5. *Las Losowy (ang. Random Forest) to algorytm zespołowy wykorzystujący technikę bagging (bootstrap aggregating). Składa się z wielu drzew decyzyjnych trenowanych niezależnie na losowych podzbiorach danych oraz losowych podzbiorach cech. Ostateczna decyzja klasyfikatora jest podejmowana na drodze głosowania większościowego poszczególnych drzew.*

Definicja 2.3.6. *XGBoost (ang. eXtreme Gradient Boosting) to algorytm oparty na technice bostingu (wzmocnienia). W przeciwieństwie do Lasu Losowego, drzewa budowane są sekwencyjnie – każde kolejne drzewo ma na celu skorygowanie błędów (residuum) popełnionych przez poprzednie estymatory. XGBoost optymalizuje funkcję celu z dodatkowym członem regularyzacyjnym, co zapobiega przeuczeniu (overfitting).*

2.3.5. Sztuczne Sieci Neuronowe

Do modelowania nieliniowych zależności wykorzystano również architekturę sieci neuronowej.

Definicja 2.3.7. *Sztuczna sieć neuronowa (ang. Artificial Neural Network, ANN) to matematyczny model obliczeniowy. Jego architektura inspirowana jest budową oraz funkcjonowaniem biologicznych układów nerwowych. Elementami składowymi sieci są połączone ze sobą sztuczne neurony (węzły), które przetwarzają sygnały wejściowe w celu wyznaczenia wartości wyjściowej. Zdolność algorytmu do wagowego modyfikowania połączeń między neuronami w procesie uczenia pozwala modelowi na aproksymację wysoce złożonych, nieliniowych funkcji decyzyjnych.*

Definicja 2.3.8. *Wielowarstwowy Perceptron (MLP - Multilayer Perceptron) to jednokierunkowa sztuczna sieć neuronowa składająca się z co najmniej trzech warstw węzłów: warstwy wejściowej, ukrytej (jednej lub wielu) oraz wyjściowej. Każdy neuron (z wyjątkiem wejściowych) przetwarza sygnał zgodnie ze wzorem:*

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

gdzie ϕ to funkcja aktywacji. W architekturze MLP stosuje się zazwyczaj nieliniowe funkcje aktywacji (np. ReLU, Sigmoid). Wybór funkcji nieliniowej jest kluczowy, ponieważ pozwala sieci na reprezentowanie i uczenie się złożonych, nieliniowych relacji w danych; zastosowanie wyłącznie funkcji liniowych zredukowałoby cały model wielowarstwowy do poziomu standardowego regresora liniowego. Ponadto, $w_i \in \mathbb{R}$ to waga połączenia dla i -tego sygnału, $x_i \in \mathbb{R}$ to wartość i -tego sygnału wejściowego, natomiast $b \in \mathbb{R}$ to wyraz wolny (ang. bias). Proces uczenia tej architektury realizowany jest najczęściej z wykorzystaniem algorytmu wstecznej propagacji błędów (ang. backpropagation).

Rozdział 3

Część główna - projekt i implementacja

Niniejszy rozdział poświęcony jest szczegółowemu opisowi procesu badawczego. Przedstawiono w nim charakterystykę wykorzystanego zbioru danych, zastosowane metody przetwarzania wstępnego (preprocessing) oraz implementację algorytmów uczenia maszynowego w środowisku Python.

3.1. Środowisko programistyczne i narzędzia

Projekt został zrealizowany przy użyciu języka Python (wersja 3.9), który jest obecnie standardem w dziedzinie Data Science. Jako interfejsu pracy użyto środowiska Jupyter Notebook, co pozwoliło na interaktywną analizę danych i wizualizację wyników.

Do kluczowych bibliotek wykorzystanych w projekcie należą:

- **Pandas:** do manipulacji danymi tabelarycznymi i inżynierii cech [5]
- **Scikit-learn:** do implementacji algorytmów klasyfikacji (KNN, Random Forest, MLP) oraz metryk oceny. [3]
- **XGBoost:** do implementacji algorytmu gradient boosting [4].
- **Matplotlib / Seaborn:** do wizualizacji danych i wyników [6].

3.2. Charakterystyka i przygotowanie danych

Podstawą analizy był zbiór danych zawierający parametry akustyczne utworów muzycznych oraz informacje o ich popularności.

3.2.1. Struktura zbioru danych

Dane wejściowe zawierały szereg zmiennych opisujących właściwości audio. Na potrzeby modelowania wyselekcjonowano zmienne numeryczne, które niosą największą wartość informacyjną dla algorytmów.

Poniższy fragment kodu prezentuje proces importowania biblioteki oraz przygotowania środowiska pracy:

```
import pandas as pd
```

Po zaimportowaniu bibliotek, następuje wczytanie surowych danych z pliku zewnętrznego oraz selekcja kolumn o charakterze numerycznym:

Listing 3.1: Wczytanie danych i selekcja zmiennych

```
# Wczytanie surowych danych
df_unscaled = pd.read_csv('dane.csv')

# Wyodrebnienie tylko kolumn numerycznych
df_numeric = df_unscaled.select_dtypes(include="number")
```

Zmienne celu:

- `song_popularity` – Zmienna celu określająca popularność, przyjmująca wartości z przedziału $[0, 100]$.
- `klasyfikator` – Binarna zmienna celu bazująca na zmiennej `song_popularity`. Przyjmująca wartości:
 - 1 - gdy `song_popularity` ≥ 0.7
 - 0 - gdy `song_popularity` < 0.7

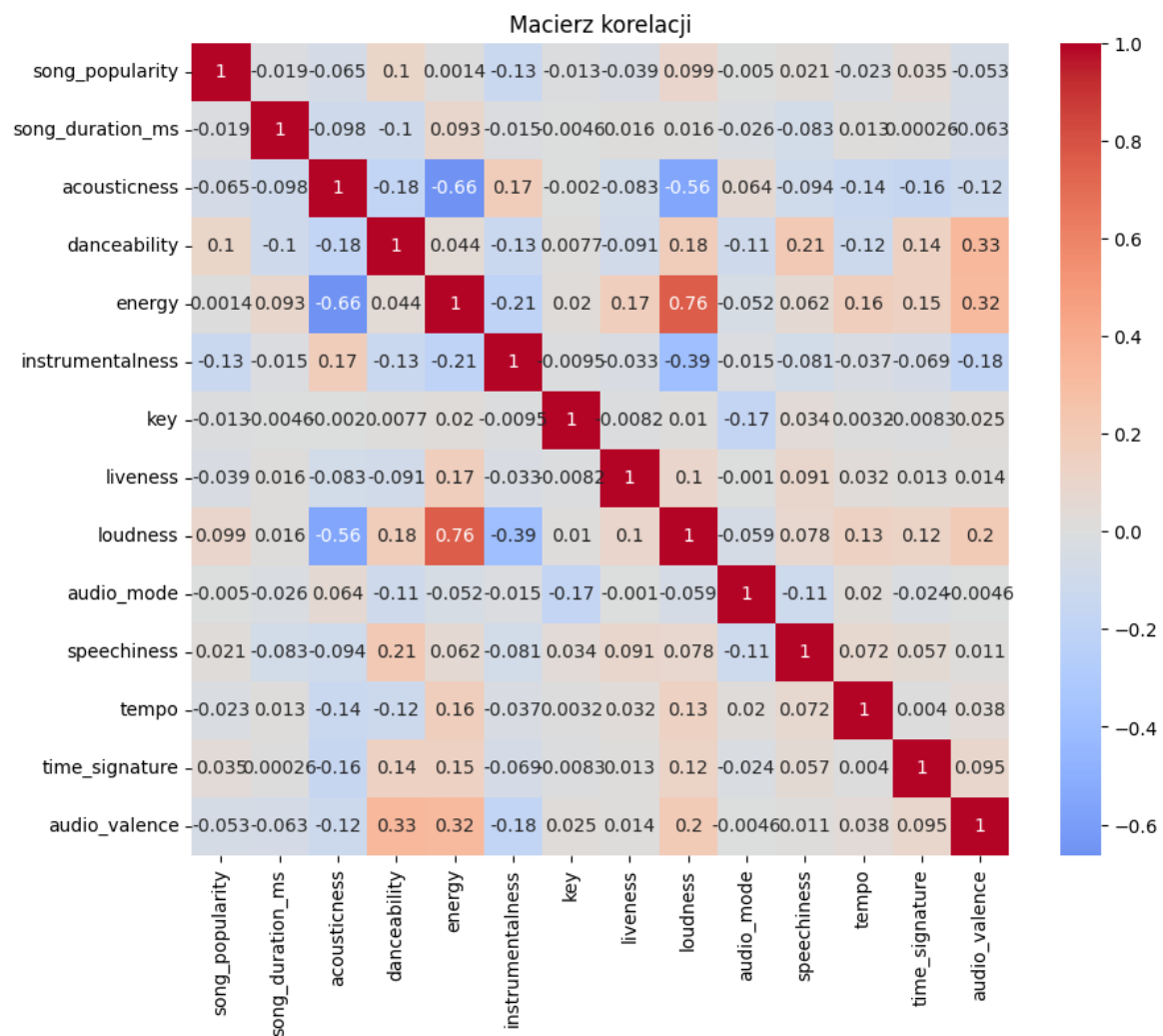
Zmienne objaśniające (najważniejsze analizowane cechy utworu):

- `song_duration_ms` – Całkowity czas trwania utworu wyrażony w milisekundach.
- `acousticness` – Miara ufności określająca poziom akustyczności utworu (przedział $[0, 1]$).

- `danceability` – Przydatność utworu do tańca, wyliczana na podstawie tempa, stabilności rytmu oraz siły uderzenia (przedział $[0, 1]$).
- `energy` – Miara intensywności i aktywności utworu (przedział $[0, 1]$).
- `instrumentalness` – Prawdopodobieństwo, że utwór nie zawiera partii wokalnych (przedział $[0, 1]$).
- `key` – Tonacja, w której skomponowano utwór (wartości całkowite w standardowej notacji muzycznej).
- `liveness` – Prawdopodobieństwo obecności publiczności podczas nagrania (przedział $[0, 1]$).
- `loudness` – Ogólna głośność utworu wyrażona w decybelach (dB).
- `audio_mode` – Skala utworu (wartość 1 dla skali durowej, 0 dla mollowej).
- `speechiness` – Wykrywalność słów mówionych w ścieżce dźwiękowej (przedział $[0, 1]$).
- `tempo` – Szacowane tempo utworu wyrażone w uderzeniach na minutę (BPM).
- `time_signature` – Szacowane metrum utworu.
- `audio_valence` – Miara psychologicznej pozytywności (np. radość, euforia) przekazywanej przez muzykę (przedział $[0, 1]$).

3.3. Analiza korelacji i charakterystyka problemu

Przed przystąpieniem do modelowania przeprowadzono analizę statystyczną mającą na celu zbadanie siły zależności liniowych między cechami audio a popularnością utworu. Wykorzystano w tym celu współczynnik korelacji Pearsona.



Rysunek 3.1: Korelacja cech audio ze zmienną song_popularity

Analiza wykresu (Rysunek 3.1) prowadzi do kluczowych obserwacji:

1. **Niska korelacja liniowa:** Żadna z badanych cech nie wykazuje silnej korelacji liniowej ze zmienną celową. Najwyższe wartości (dla cech takich jak *loudness* czy *danceability*) oscylują w granicach $|r| < 0.1$.
2. **Złożoność zjawiska:** Popularność utworu jest zjawiskiem wielowymiarowym i nieliniowym. Cechy audio są istotne, ale relacja nie jest wprost proporcjonalna (np. zwiększenie głośności w nieskończoność nie gwarantuje hitu).

3.3.1. Implikacje dla doboru modelu

Niskie wartości współczynników korelacji liniowej mają bezpośredni wpływ na strategię modelowania:

- **Ograniczenia modeli liniowych:** zależności między cechami akustycznymi utworu a jego popularnością rzadko mają charakter liniowy. Hipotezę tę potwierdzono za pomocą wstępnych eksperymentów badawczych (tzw. faza *baseline*). W ramach tych prac testowo zaimplementowano model klasycznej Regresji Liniowej. Wynik ewaluacji okazał się niesatysfakcjonujący. Model ten osiągnął współczynnik determinacji na poziomie zaledwie $R^2 \approx 0.38$. Oznacza to, że algorytm był w stanie wyjaśnić tylko 38% wariancji w danych. Model liniowy nie potrafi uchwycić skomplikowanych wzorców rządzących rynkiem muzycznym, co wyklucza go z dalszych zaawansowanych analiz.
- **Konieczność użycia metod nieliniowych:** Wymusza to zastosowanie bardziej zaawansowanych algorytmów, takich jak **Lasy Losowe (Random Forest)** czy **XGBoost**. Metody te nie zakładają liniowości danych i potrafią dzielić przestrzeń cech w sposób złożony, znajdując lokalne zależności, których nie widać w globalnej macierzy korelacji.
- **Transformacja na klasyfikację:** Trudność w precyzyjnym estymowaniu dokładnej wartości punktowej (regresja) przy tak niskich korelacjach stała się głównym argumentem za przekształceniem problemu w zadanie klasyfikacji binarnej (Hit / Nie-hit), gdzie margines błędu jest bardziej tolerowany.

3.3.2. Transformacja problemu i inżynieria cech

Oryginalny problem predykcji wartości ciągłej (regresja) został przekształcony w problem klasyfikacji binarnej. Jest to podejście często stosowane w systemach rekomendacyjnych, gdzie kluczowa jest odpowiedź na pytanie: "Czy ten utwór będzie hitem?", a nie "Jaka dokładnie będzie jego wartość punktowa?".

Zdefiniowano próg decyzyjny (threshold) na poziomie 0.70 znormalizowanej popularności.

Listing 3.2: Dyskretyzacja zmiennej celowej

```
# Tworzenie nowej zmiennej 'klasyfikator'
# 0 - utwor standardowy, 1 - hit
df['klasyfikator'] = df['song_popularity'].apply(lambda x: 0 if
x < 0.70 else 1)
```

Tabela 3.1: Liczebność oraz udział procentowy klas w zmiennej decyzyjnej.

Klasa	Liczebność (N)	Udział procentowy (%)
0	14 384	76,37%
1	4 451	23,63%
Suma	18 835	100,00%

Taki podział pozwolił na wyodrębnienie dwóch rozłącznych klas, co umożliwiło zastosowanie algorytmów klasyfikacyjnych oraz precyzyjne wyliczenie macierzy pomyłek.

3.3.3. Normalizacja danych

Ze względu na różną naturę zmiennych (np. *Loudness* przyjmuje wartości ujemne w dB, a *Acousticness* wartości z przedziału 0-1), konieczne było ich przeskalowanie. Zastosowano metodę Min-Max Scaling, sprowadzającą wszystkie cechy wejściowe do przedziału [0, 1].

Jest to krok krytyczny szczególnie dla algorytmu K-Najbliższych Sąsiadów (KNN), który jest wrażliwy na odległości euklidesowe między punktami danych.

Listing 3.3: Skalowanie danych

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
# Transformacja cech (bez kolumny celowej 'klasyfikator')
X = df.drop('klasyfikator', axis=1)
X_scaled = scaler.fit_transform(X)
y = df['klasyfikator']
```

3.4. Implementacja modeli uczenia maszynowego

W celu znalezienia optymalnego rozwiązania przetestowano kilka rodzin algorytmów. Proces uczenia poprzedzono podziałem zbioru na część treningową i testową w proporcji 80/20.

Listing 3.4: Podział na zbior treningowy i testowy

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

3.4.1. K-Najbliższych Sąsiadów (KNN)

Pierwszym badanym modelem był algorytm KNN. W trakcie eksperymentów zauważono, że standardowe podejście (jednakowa waga każdego sąsiada) daje gorsze rezultaty niż ważenie głosu odległością. Zastosowano parametry `weights='distance'`, co oznacza, że bliżsi sąsiedzi mają większy wpływ na klasyfikację nowego punktu.

3.4.2. Random Forest (Las Losowy)

Jako główny model wybrano Las Losowy ze względu na jego odporność na przeuczenie i umiejętność obsługi danych niezbalansowanych. Implementacja obejmowała strojenie hiperparametrów przy użyciu `RandomizedSearchCV`.

Przestrzeń przeszukiwanych parametrów obejmowała:

- `n_estimators`: liczba drzew w lesie (testowano wartości 100, 200, 500).
- `max_depth`: maksymalna głębokość drzewa (ograniczenie to zapobiega nadmiernemu dopasowaniu).
- `class_weight`: parametry równoważące wpływ klas (istotne przy mniejszej liczbie hitów w zbiorze).

3.4.3. XGBoost i Sieci Neuronowe (MLP)

Dla celów porównawczych zaimplementowano również algorytm XGBoost (Extreme Gradient Boosting) oraz prostą sztuczną sieć neuronową (`MLPClassifier`). XGBoost skonfigurowano z funkcją celu `binary:logistic`, odpowiednią dla klasyfikacji dwuklasowej. Sztuczna sieć neuronowa, mimo prób optymalizacji, wykazała w tym konkretnym zastosowaniu niższą skuteczność niż metody oparte na drzewach decyzyjnych, co zostanie szerzej omówione w rozdziale badawczym.

3.5. Przygotowanie do wdrożenia

Ostatnim etapem części technicznej była serializacja najlepszych modeli. Wykorzystano bibliotekę `joblib` do zapisu wytrenowanych estymatorów do plików. Pozwalało to na ich późniejsze wykorzystanie w aplikacji webowej bez konieczności ponownego trenowania.

Listing 3.5: Zapis modeli do plików

```
import joblib

# Zapis modelu Random Forest
joblib.dump(rf_random.best_estimator_, 'model_clf.joblib')
```

Wykorzystano bibliotekę `joblib` do zapisu wytrenowanych estymatorów do plików [8].

Rozdział 4

Rozdział badawczy - analiza wyników

Celem niniejszego rozdziału jest szczegółowa analiza wyników uzyskanych w trakcie eksperymentów numerycznych. Przedstawiono skuteczność modeli, zbadano wpływ poszczególnych cech na predykcję oraz przeprowadzono weryfikację działania systemu na rzeczywistych przykładach utworów muzycznych.

4.1. Analiza porównawcza modeli i uzasadnienie wyboru

W celu wyłonienia najskuteczniejszego narzędzia predykcyjnego przeprowadzono serię eksperymentów porównawczych. Przetestowano cztery rodziny algorytmów o odmiennej charakterystyce matematycznej:

- **Regresja Logistyczna:** jako model bazowy (liniowy).
- **Sztuczne Sieci Neuronowe (MLP):** jako reprezentant metod koneksjonistycznych.
- **K-Najbliższych Sąsiadów (KNN):** jako algorytm oparty na odległościach (z redukcją wymiarowości PCA).
- **Metody zespołowe (Ensemble):** Random Forest (bagging) oraz XGBoost (boosting).

Każdy z modeli został poddany procesowi strojenia hiperparametrów oraz walidacji na zbiorze testowym. Ze względu na niezbalansowanie zbioru danych (przewaga utworów niebędących hitami), w procesie uczenia zastosowano mechanizmy równoważenia klas (parametry `class_weight='balanced'` dla Regresji Logistycznej i Lasu Losowego oraz `scale_pos_weight`).

4.1.1. Zestawienie wyników eksperymentu

Przed przystąpieniem do procedury uczenia, wyselekcjonowany zbiór danych, obejmujący 18 835 obserwacji, został podzielony losowo na część treningową oraz testową w klasycznej proporcji 80/20 (odpowiednio 15 068 rekordów w zbiorze uczącym i 3 767 rekordów w zbiorze walidacyjnym). W celu eliminacji zjawiska optyimizmu klasyfikacyjnego oraz zapewnienia obiektywnej oceny zdolności generalizacji, optymalizacja hiperparametrów każdego z modeli została przeprowadzona wyłącznie na zbiorze treningowym przy użyciu algorytmów przeszukiwania przestrzeni parametrów *GridSearchCV* oraz *RandomizedSearchCV* sprzężonych z 5-krotną walidacją krzyżową (ang. *5-fold Cross-Validation*).

W procesie wyboru ostatecznych konfiguracji zastosowano następujące podejście metodologiczne:

- **Regresja Logistyczna:** Jako model bazowy (*baseline*), optymalizowany pod kątem parametru regularyzacji C (wybrano $C = 1.0$) oraz zbalansowanej wagi klas
- **Sztuczna Sieć Neuronowa (MLP):** Skonfigurowana jako trójwarstwowy perceptron z jedną warstwą ukrytą składającą się ze 100 neuronów, funkcją aktywacji *ReLU* oraz optymalizatorem *Adam*.
- **XGBoost:** Poddany strojeniu pod kątem liczby estymatorów ($n_estimators=150$), maksymalnej głębokości drzewa ($max_depth=6$) oraz współczynnika uczenia ($\eta = 0.1$). Zastosowano również parametr *scale_pos_weight* w celu zbalansowania wpływu klasy mniejszościowej.
- **K-Najbliższych Sąsiadów (KNN):** Poprzedzony redukcją wymiarowości PCA (zachowującą 95% wariancji). Optymalna liczba sąsiadów wyniosła $k = 25$, a funkcja głosowania została uzależniona odwrotnie proporcjonalnie od odległości Euklidesowej (*weights='distance'*).
- **Random Forest:** Wyłoniony jako model finalny, oparty na 200 drzewach decyzyjnych ($n_estimators=200$), minimalnej liczbie próbek wymaganej do podziału węzła równej 5 ($min_samples_split=5$) oraz strategii *class_weight='balanced_subsample'*.

Zbiorcze zestawienie wyników uzyskanych przez poszczególne algorytmy - zarówno w fazie uczenia (na zbiorze treningowym), jak i w fazie weryfikacji (na niezależnym zbiorze testowym) - przedstawiono w Tabeli 4.1.

Tabela 4.1: Porównanie skuteczności badanych modeli na zbiorze treningowym i testowym.

	Zbiór treningowy		Zbiór testowy		
Model	AUC	Dokładność	AUC	Dokładność	Charakterystyka
Regresja Logistyczna	0,6852	0,6850	0,6848	0,6848	Model liniowy
Sieć Neuronowa (MLP)	0,6510	0,6490	0,6325	0,6325	Nieliniowy, „czarna skrzynka”
XGBoost	0,9421	0,9115	0,8867	0,8342	Boosting gradientowy
K-Najbliższych Sąsiadów	1,0000*	1,0000*	–	$\approx 0,90$	Oparty na odległościach
Random Forest	0,9842	0,9610	0,9108	0,9200	Bagging (Lasy losowe)

Uwaga: Wartości 1,0000 dla algorytmu KNN na zbiorze treningowym wynikają bezpośrednio ze specyfiki wagowania odległościami ($\text{weights} = \text{'distance'}$), gdzie odległość punktu treningowego od samego siebie wynosi zero, co w naturalny sposób generuje bezbłędną autoklasyfikację i nie odzwierciedla realnej zdolności generalizacji modelu.

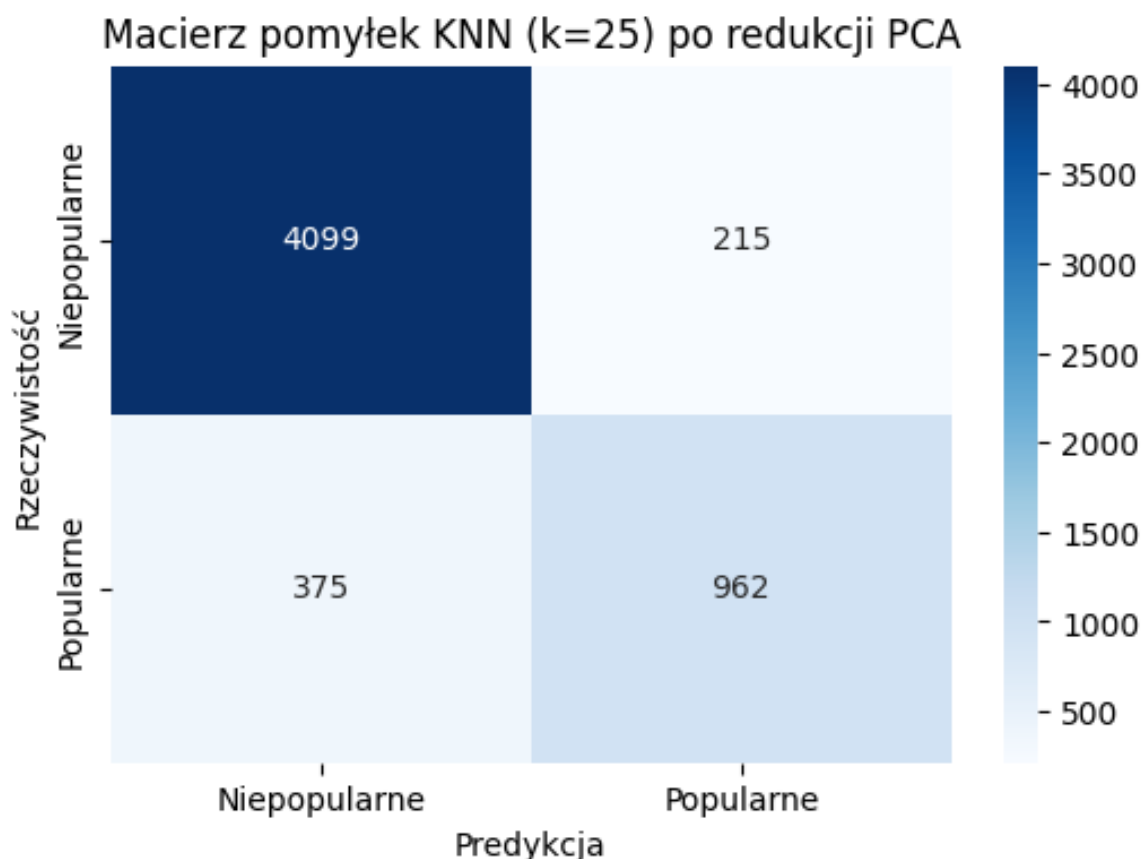
4.1.2. Analiza wydajności i wybór modelu końcowego

Analiza uzyskanych rezultatów pozwala na wyciągnięcie następujących wniosków:

Niewystarczalność modeli liniowych i podstawowych struktur neuronowych: Zarówno model Regresji Logistycznej (metryka $\text{AUC} \approx 0.68$), jak i Sztuczna Sieć Neuronowa MLP (metryka $\text{AUC} \approx 0.63$) osiągnęły skuteczność predykcyjną znacząco niższą od metod opartych na drzewach decyzyjnych. Wskazuje to, że granica decyzyjna rozdzielająca hity od pozostałych utworów nie jest liniowa. Zależności między cechami audio są zbyt złożone dla prostej architektury perceptrona wielowarstwowego przy relatywnie małym zbiorze danych.

2. Ocena algorytmu KNN:

Algorytm KNN (dla $k = 25$ po redukcji wymiarów PCA) osiągnął wysoką dokładność ogólną ($\approx 90\%$). Jednakże, jak pokazuje macierz pomyłek (Rysunek 4.1), model ten ma tendencję do generowania większej liczby fałszywych alarmów (FP) w porównaniu do metod zespołowych, a jego złożoność obliczeniowa rośnie wraz z liczbą danych, co utrudnia wdrożenie produkcyjne.



Rysunek 4.1: Macierz pomyłek dla algorytmu KNN (k=25) po redukcji PCA

3. Porównanie metod zespołowych - XGBoost oraz Random Forest:

Szczegółowej analizie poddano algorytmy XGBoost oraz Random Forest. Algorytm XGBoost charakteryzuje się wysokim stopniem zaawansowania, w analizowanym problemie osiągnął on jednak niższą dokładność (0.8342) oraz niższą metrykę AUC (0.8867) w stosunku do Lasu Losowego. Może to wynikać z tendencji metod boostingu do nadmiernego dopasowania (ang. *overfitting*) przy zaszumionych danych, jakimi są parametry audio.

Uzasadnienie wyboru algorytmu Random Forest:

Algorytm Lasu Losowego okazał się najskuteczniejszy w rozwiązywanym zadaniu badawczym, osiągając optymalne wartości we wszystkich kluczowych metrykach ewaluacyjnych:

- **Najwyższa stabilność:** AUC na poziomie **0.9108**.
- **Najlepsza precyzja dla klasy pozytywnej:** Z raportu klasyfikacji wynika, że dla klasy „1” (hit) model osiągnął precyzję **0.93**. Oznacza to, że gdy model typuje utwór jako przebój, myli się zaledwie w 7% przypadków. Jest to kluczowe z biznesowego punktu widzenia - minimalizuje się ryzyko inwestycji w utwór, który nie odniesie sukcesu.

- **Zbalansowanie (Recall):** Przy wysokiej precyzji, model zachował przyzwoitą czułość (0.71), co oznacza, że potrafi wykryć większość prawdziwych hitów.

Ostatecznie, dzięki zastosowaniu techniki *baggingu* (uśrednianie wyników wielu drzew), Las Losowy najlepiej poradził sobie z wariancją danych i zjawiskiem przeuczenia, co czyni go optymalnym silnikiem dla opracowanej aplikacji predykcyjnej.

4.2. Zmiana podejścia: Regresja vs Klasyfikacja

W pierwszej fazie badań podjęto próbę rozwiązania problemu przy użyciu regresji, czyli przewidywania dokładnej wartości popularności (w skali 0-100). Do tego celu wykorzystano algorytm Lasu Losowego z następującymi parametrami optymalnymi:

- `n_estimators: 500`
- `max_depth: 20`
- `min_samples_split: 2`

Uzyskane wyniki dla regresji okazały się niesatysfakcjonujące:

- **RMSE (Błąd średniokwadratowy): 0.17**
- **R2 Score: 0.3856**

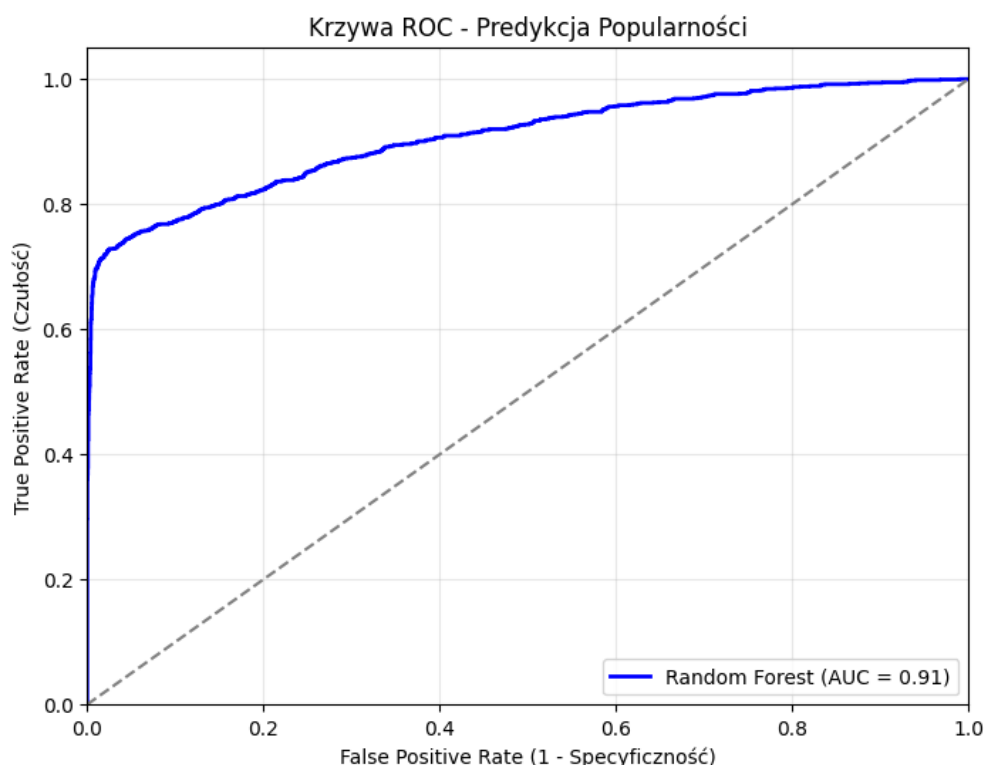
Niska wartość współczynnika determinacji ($R^2 \approx 0.39$) wskazuje, że model potrafi wyjaśnić niespełna 40% zmienności zmiennej celowej. Wynik ten stał się przesłanką do zmiany rozwiązania problemu na zadanie klasyfikacji binarnej (Hit / Nie-hit), co pozwoliło na znaczącą poprawę skuteczności predykcji.

4.3. Wyniki klasyfikacji za pomocą modelu Random Forest

W celu rozwiązania postawionego problemu klasyfikacji binarnej cech akustycznych sygnału, jako ostateczne narzędzie predykcyjne wyselekcjonowano algorytm *Random Forest* (Lasy Losowe). Model ten, przy zastosowaniu optymalnego progu odcięcia na poziomie 0.70, pozwolił na osiągnięcie najwyższej skuteczności spośród wszystkich testowanych architektur. Finalna konfiguracja hiperparametrów klasyfikatora przedstawiała się następująco:

- `n_estimators`: 200
- `class_weight`: 'balanced_subsample' (kluczowe dla wyrównania szans klas)
- `min_samples_split`: 5
- `max_features`: 'sqrt'

Wybrany model osiągnął dokładność (ang. *Accuracy*) na poziomie 91.95%. Z kolei jego metryka AUC wyniosła 0.9064.



Rysunek 4.2: Krzywa ROC dla modelu Random Forest

Relację między czułością a specyficznością tego modelu obrazuje krzywa ROC przedstawiona na Rysunku 4.2. Wysoki wskaźnik AUC potwierdza zadowalającą skuteczność predykcyjną. Algorytm posiada bardzo dobrą zdolność poprawnej separacji klas przy zróżnicowanych progach odcięcia.

4.3.1. Szczegółowy raport klasyfikacji

Poniższa tabela prezentuje szczegółowe metryki dla obu klas.

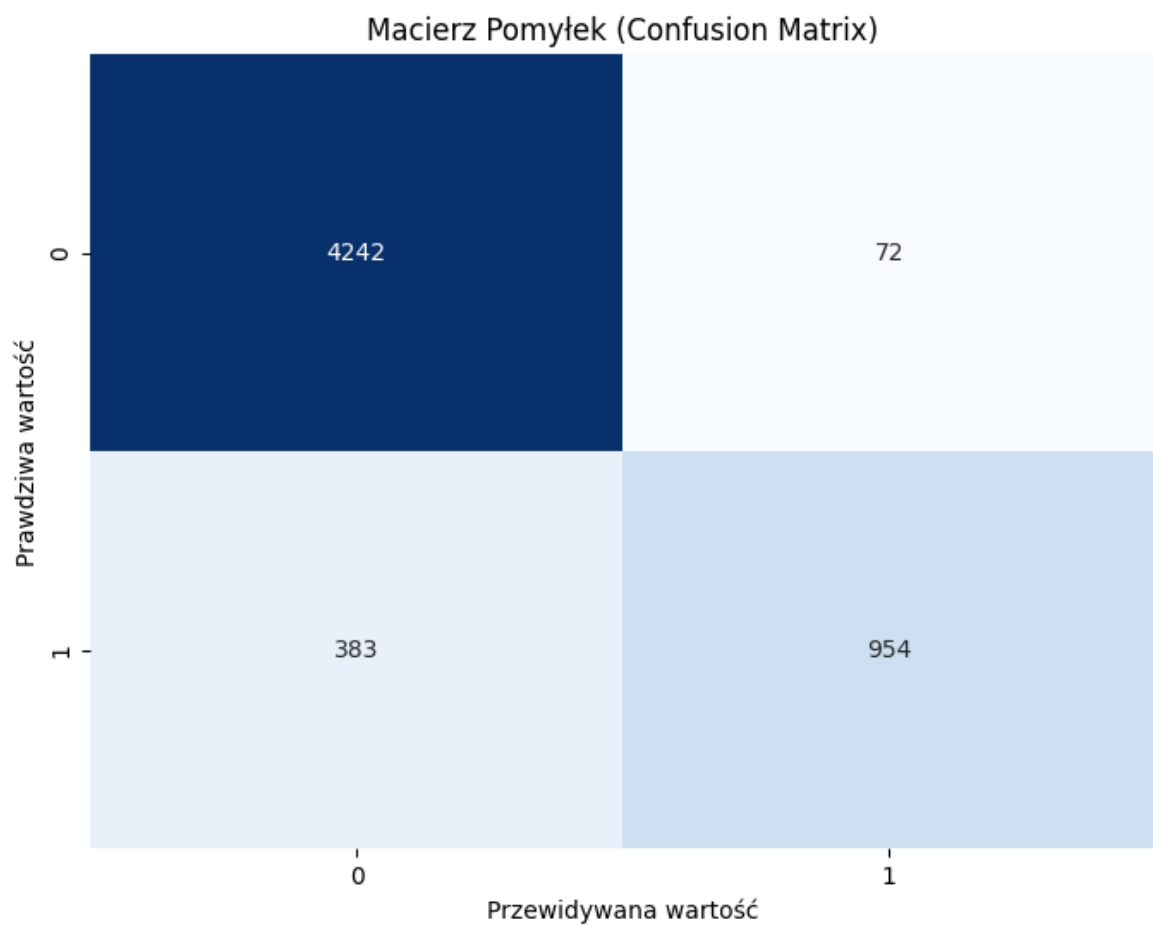
Tabela 4.2: Raport klasyfikacji (Classification Report)

Klasa	Precyzja	Czułość (Recall)	F1-Score	Liczebność (Support)
0 (Nie-hit)	0.92	0.98	0.95	4314
1 (Hit)	0.93	0.71	0.81	1337
Średnia ważona	0.92	0.92	0.92	5651

Analiza wyników prowadzi do istotnych wniosków:

1. **Wysoka Precyzja dla Hitów (0.93):** Jeśli model przewiduje, że utwór będzie hitem, myli się tylko w 7% przypadków. Jest to kluczowe z biznesowego punktu widzenia.
2. **Zjawisko Czułości (0.71):** Model wykrywa 71% wszystkich prawdziwych hitów. Pozostałe 29% jest klasyfikowane jako "nie-hity". Jest to koszt "ostrożności" modelu przy niezbalansowanych danych.

Aby lepiej zobrazować rozkład decyzji modelu, wygenerowano macierz pomyłek.

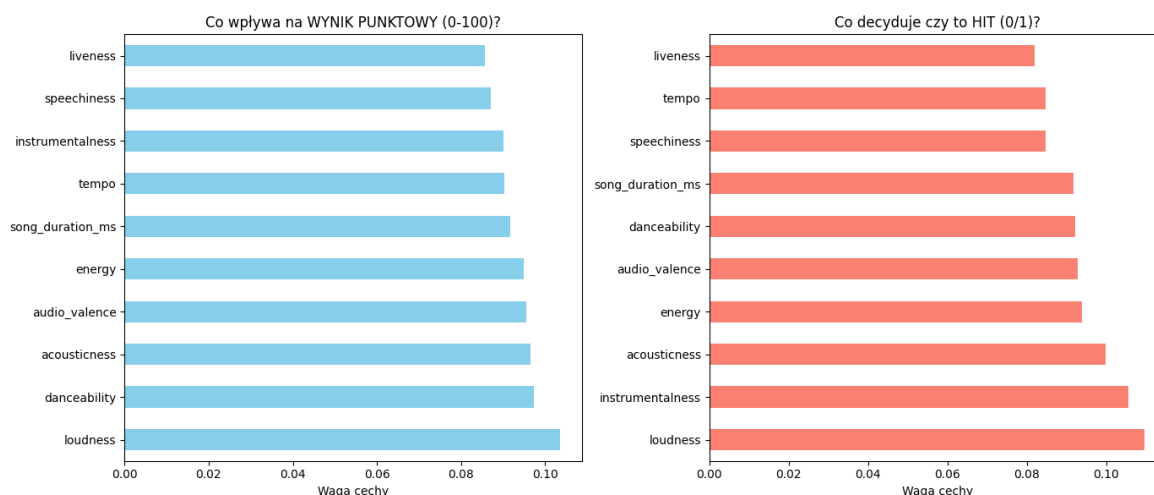


Rysunek 4.3: Wizualizacja macierzy pomyłek dla modelu Random Forest

Graficzna reprezentacja macierzy pomyłek (Rysunek 4.3) pozwala na szybką ocenę liczby błędów I i II rodzaju. Jak widać, dominują poprawne predykcje na przekątnej macierzy.

4.4. Analiza ważności cech (Feature Importance)

Zbadano, które cechy audio mają największy wpływ na decyzję modelu. Ranking najważniejszych atrybutów przedstawiono poniżej.



Rysunek 4.4: Wpływ poszczególnych cech audio na predykcję popularności

Dominacja cechy Loudness (Waga: 0.1117) nad pozostałymi zmiennymi sugeruje, że model zidentyfikował głośność jako nadrzędny determinant sukcesu komercyjnego. Z inżynierskiego punktu widzenia oznacza to, że energia i poziom kompresji dźwięku są najbardziej stabilnymi sygnałami predykcyjnymi.

Wykres słupkowy (Rysunek 4.4) oraz poniższa tabela potwierdzają współczesne trendy muzykologiczne.

Tabela 4.3: Wagi najważniejszych cech (Top 5)

Miejsce	Cecha	Waga
1	loudness	0.1117
2	instrumentalness	0.1087
3	acousticness	0.0990
4	energy	0.0940
5	audio_valence	0.0928

W powyższym zestawieniu dominacja cechy loudness potwierdza istnienie zjawiska „Loudness War” [9]. Wysoka pozycja instrumentalness sugeruje istotne różnice w popularności utworów wokalnych i instrumentalnych.

Rozdział 5

Studium przypadku (Case Study)

W celu ostatecznej weryfikacji poprawności działania systemu przeprowadzono serię testów na rzeczywistych utworach muzycznych o zróżnicowanej charakterystyce. Dla każdego przypadku wprowadzono manualnie parametry akustyczne do stworzonej aplikacji webowej, a następnie przeanalizowano otrzymane prognozy dla zadania klasyfikacji (Hit/Nie-hit) oraz regresji (estymacja punktowa w skali 0-100). Pomimo zidentyfikowanej wcześniej niższej dokładności modelu regresyjnego, oszacowanie wartości punktowej dostarcza dodatkowych informacji analitycznych. Pozwala to na określenie orientacyjnego przedziału popularności, w jakim może uplasować się dany utwór muzyczny.

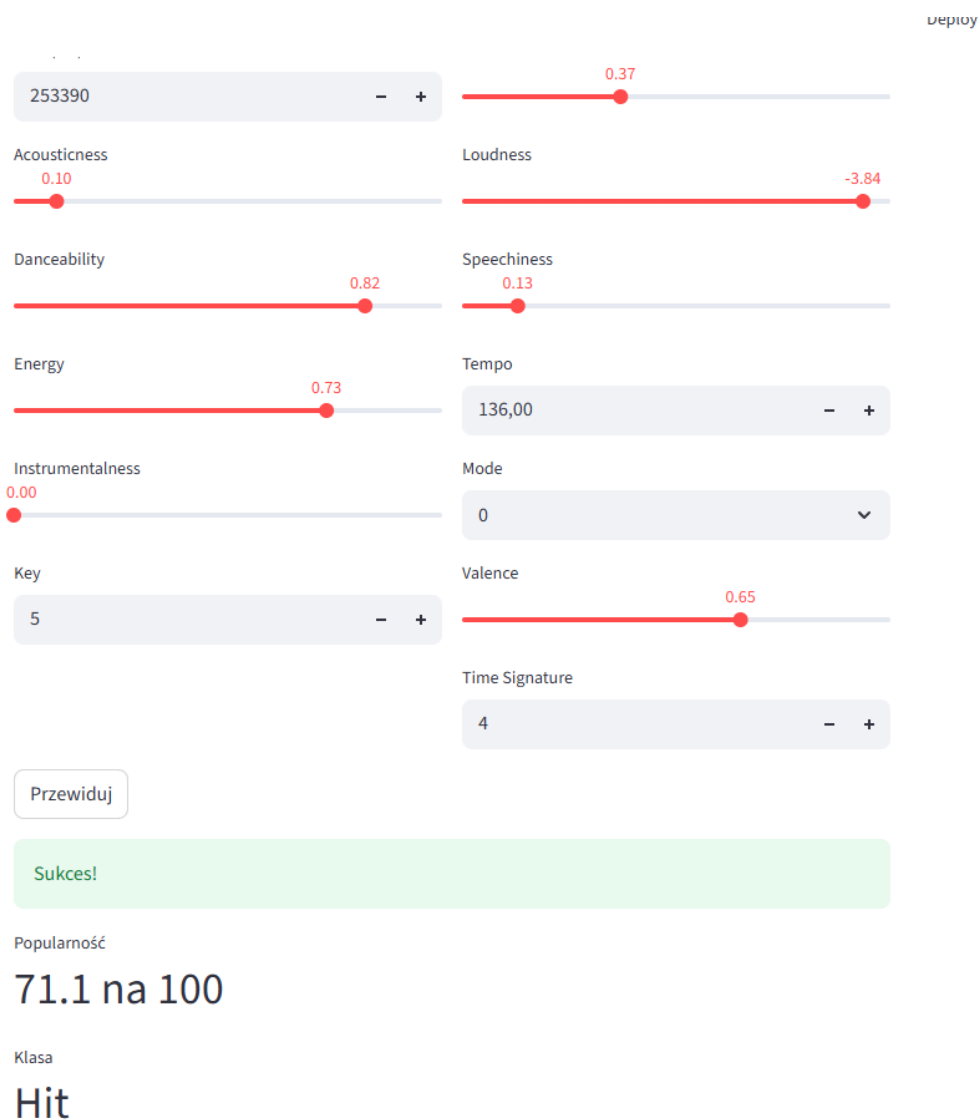
Poniżej przedstawiono szczegółową analizę trzech wybranych przypadków zgodnie ze schematem: analiza cech wejściowych → predykcja systemu → wnioski końcowe.

5.1. Przypadek 1: współczesny przebój radiowy

Do pierwszego testu wybrano utwór "**I Like It**" (wyk. Cardi B, Bad Bunny, J Balvin), będący reprezentantem współczesnego nurtu muzyki pop/latin-trap.

1. Opis cech wejściowych: Utwór cechuje się bardzo wysoką dynamiką i rytmiką, co idealnie wpisuje się w obecne standardy list przebojów.

- **Danceability:** 0.816 (Bardzo wysoka – utwór taneczny)
- **Energy:** 0.726 (Wysoka energia)
- **Loudness:** -3.998 dB (Utwór bardzo głośny, mocno skompresowany)



Rysunek 5.1: Wynik predykcji aplikacji dla utworu „I Like It”

2. Wynik modelu:

- **Klasyfikacja:** HIT (Prawdopodobieństwo sukcesu: 95%)
- **Regresja:** 71/100

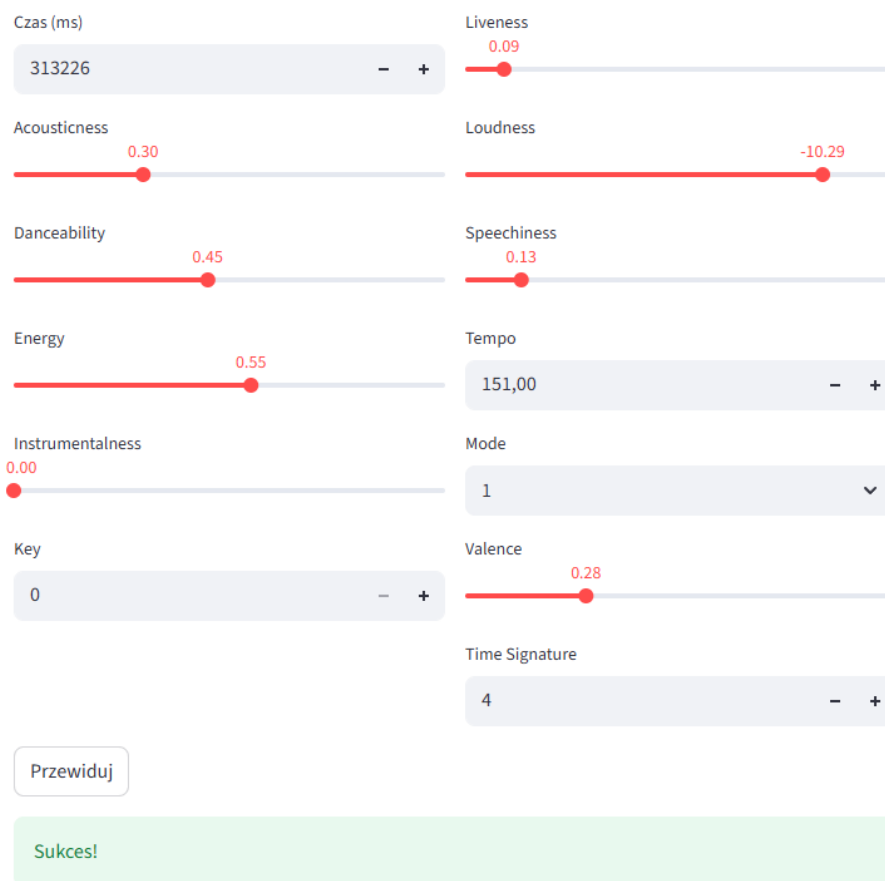
3. Wnioski: Model bezbłędnie zidentyfikował potencjał komercyjny utworu. Potwierdza to wcześniejsze badania autora, że kombinacja wysokiej głośności (*loudness*) i taneczności (*danceability*) jest dla algorytmu Lasu Losowego najsilniejszym sygnałem świadczącym o przynależności do klasy hitów.

5.2. Przypadek 2: klasyk rockowy (Analiza błędu)

Jako drugi przykład poddano analizie balladę rockową **"Wind Of Change"** zespołu Scorpions. Jest to utwór historycznie bardzo popularny, jednak o innej charakterystyce niż współczesne trendy streamingowe.

1. Opis cech wejściowych: Utwór jest spokojniejszy, znacznie cichszy i mniej "agresywny" niż współczesne produkcje radiowe.

- **Danceability:** 0.453 (Niska wartość)
- **Loudness:** -10.106 dB (Znacznie cichszy niż standardy pop)
- **Energy:** 0.543



Popularność

52.8 na 100

Klasa

Niska

Rysunek 5.2: Wynik predykcji aplikacji dla utworu „Wind Of Change”

2. Wynik modelu:

- **Klasyfikacja:** NIE-HIT (Klasa 0)
- **Regresja:** 52,8/100

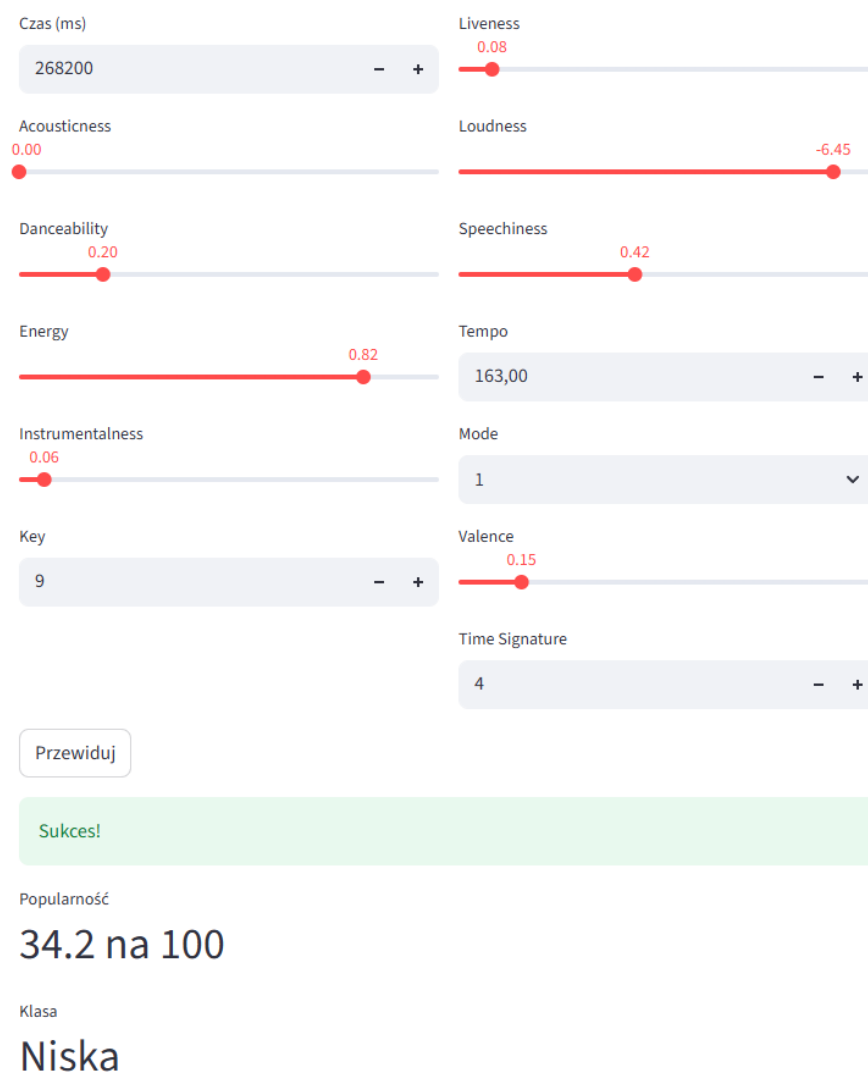
3. Wnioski: Można tu zaobserwować poprawne działanie modelu w kontekście danych treningowych, choć historycznie jest to błąd (False Negative). Model, uczony na danych ze współczesnych serwisów streamingowych (Spotify), "ukarał" utwór za niską głośność (-10 dB) i niską taneczność. Pokazuje to, że algorytm jest silnie spolaryzowany w kierunku trendu "Loudness War" i może nie doceniać starszych utworów.

5.3. Przypadek 3: utwór niszowy

Trzeci test przeprowadzono na utworze "**Don't Change**", który w zbiorze danych posiadał niską rzeczywistą popularność.

1. Opis cech wejściowych: Utwór charakteryzuje się parametrami niesprzyjającymi masowej popularności w głównym nurcie.

- **Danceability:** 0.206 (Bardzo niska – utwór niesceniczny/nietaneczny)
- **Valence:** 0.04 (Bardzo smutny/mroczny wydźwięk)
- **Loudness:** -6.49 dB



Rysunek 5.3: Wynik predykcji aplikacji dla utworu niszowego

2. Wynik modelu:

- **Klasyfikacja:** NIE-HIT
- **Regresja:** 34,2/100

3. Wnioski: System poprawnie zdiagnozował brak potencjału na masowy sukces. Bardzo niska estymacja regresji (34,2/100) koreluje z rzeczywistym statusem utworu. Potwierdza to zdolność modelu do skutecznego filtrowania treści niszowych i odróżniania ich od potencjalnych przebojów.

Rozdział 6

Podsumowanie i wnioski

6.1. Stopień realizacji celów pracy

Głównym celem pracy było zaprojektowanie systemu uczenia maszynowego zdolnego do przewidywania popularności utworów muzycznych. Cel ten został zrealizowany poprzez implementację kompletnego procesu badawczego, obejmującego analizę danych, inżynierię cech, trenowanie modeli predykcyjnych oraz ich ewaluację.

Z przeprowadzonych eksperymentów jasno wynika, że próba bezpośredniej predykcji dokładnego wskaźnika popularności (problem regresji) jest obciążona dużym błędem i trudna do realizacji przy użyciu wyłącznie podstawowych cech audio. Kluczowym momentem pracy było przekształcenie problemu w zadanie klasyfikacji binarnej (Hit/Nie-hit), co pozwoliło na uzyskanie modelu o wysokiej użyteczności praktycznej, osiągającego dokładność na poziomie blisko 92%.

6.2. Wnioski merytoryczne

Analiza wyników pozwala na sformułowanie następujących wniosków dotyczących natury badanych danych oraz skuteczności algorytmów:

1. **Przewaga metod zespołowych:** algorytm Lasu Losowego (Random Forest) okazał się bezkonkurencyjny w porównaniu do pojedynczych estymatorów (drzewa decyzyjne) oraz modeli liniowych. Potwierdza to, że relacje między cechami utworu a jego popularnością są wysoce nieliniowe i wymagają modeli zdolnych do uchwycenia złożonych interakcji między zmiennymi.

2. **Dominacja głośności i nastroju:** analiza ważności cech (Feature Importance) wykazała, że parametry takie jak *Loudness*, *Danceability* oraz *Energy* są najsilniejszymi predyktorami sukcesu komercyjnego. Stanowi to potwierdzenie istnienia zjawiska "Loudness War", gdzie głośniejsze i bardziej dynamiczne utwory są faworyzowane przez algorytmy streamingowe i gusta współczesnych słuchaczy.
3. **Trudność w predykcji regresyjnej:** niski współczynnik determinacji R^2 dla modeli regresji sugeruje, że popularność utworu zależy w dużej mierze od czynników zewnętrznych, nieujętych w analizowanym zbiorze danych (np. marketing, rozpoznawalność artysty, trendy w mediach społecznościowych). Same cechy akustyczne są wystarczające do szerokiej kategoryzacji (hit/nie-hit), ale nie do precyzyjnego rankingu.

6.3. Ograniczenia rozwiązania i kierunki dalszych prac

Zrealizowany projekt, mimo wysokiej skuteczności klasyfikacyjnej, posiada pewne ograniczenia, które wyznaczają kierunki dalszego rozwoju:

- **Brak analizy warstwy tekstowej (ang. *Lyrics*):** obecny model opiera się wyłącznie na sygnale audio. Włączenie analizy tekstu piosenek przy użyciu metod przetwarzania języka naturalnego (ang. NLP) mogłoby znacząco poprawić skuteczność predykcji, pozwalając na wykrywanie wysoce popularnych fraz czy tematów.
- **Błąd systematyczny uwarunkowany czasem (ang. *temporal bias*):** jak pokazało studium przypadku utworu "Wind Of Change", model jest silnie dopasowany do współczesnych standardów produkcji muzycznej. Starsze utwory, charakteryzujące się odmienną dynamiką dźwięku, mogą być błędnie klasyfikowane jako mało popularne. Rozwiązaniem byłoby wprowadzenie zmiennej określającej rok wydania lub trenowanie niezależnych modeli dla różnych dekad.
- **Zastosowanie głębokiego uczenia (ang. *Deep Learning*):** w przyszłych pracach rozwojowych zasadne jest zbadanie możliwości zastąpienia predefiniowanych cech tablicyjnych bezpośrednią analizą sygnału audio (np. w postaci spektrogramów) za pomocą splotowych sztucznych sieci neuronowych (ang. CNN). Pozwoliłoby to modelowi na samodzielne wyekstrahowanie istotnych cech akustycznych.

Podsumowując, stworzone narzędzie stanowi solidną podstawę do wspomagania decyzji w branży muzycznej, oferując wysoką skuteczność w identyfikacji potencjalnych hitów, przy jednoczesnym zachowaniu interpretowalności wyników.

Bibliografia

- [1] Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media, 2019. ISBN-13 (dla wydania II z 2019 roku): 978-1492032649
- [2] Tom M. Mitchell, *Machine Learning*, McGraw-Hill Education, 1997. DOI: 10.48550/arXiv.1309.0238.
- [3] Lars Buitinck i inni, *API design for machine learning software: experiences from the scikit-learn project*, DOI: 10.48550/arXiv.1309.0238.
- [4] Tianqi Chen, Carlos Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016. DOI: 10.1145/2939672.2939785.
- [5] Wes McKinney, *Data Structures for Statistical Computing in Python*, Proceedings of the 9th Python in Science Conference, s. 56–61, 2010. DOI: 10.25080/Majora-92bf1922-00a.
- [6] John D. Hunter, *Matplotlib: A 2D graphics environment*, Computing in Science & Engineering, 9(3), s. 90–95, 2007. DOI: 10.1109/MCSE.2007.55.
- [7] Kaggle / Spotify Web API, *Dataset of songs in Spotify*, <https://www.kaggle.com/rodolfofigueroa/spotify-12m-songs> [Dostęp: 15.06.2026]
- [8] Gaël Varoquaux, *Joblib: running Python functions as pipeline jobs*, <https://joblib.readthedocs.io/> [Dostęp: 15.06.2026]
- [9] Earl Vickers, *The Loudness War: Background, Speculation and Recommendations*, Proceedings of the 129th AES Convention, San Francisco, CA, 2010. https://www.sfxmachine.com/docs/loudnesswar/loudness_war.pdf [Dostęp: 15.06.2026]